DSA0502 – Query Processing for Data Science

CO3–AT3–DATASET INTEGRATION TASK

Course Faculty: Dr. B .SHYAMALA GOWRI **Reg. No.:** 192324164

Student Name: Sweety Roselin A **Date:** 13/08/2026

QUESTION 2 – EMPLOYEE DATASET INTEGRATION

Question

An organization has employee data stored in HR, Payroll, and Attendance datasets. The datasets contain different column names, inconsistent employee names, duplicate employees, invalid email IDs, different date formats, missing department values, and salary inconsistencies.

Task: Develop a Python program to integrate the three datasets. Apply data formatting, RegEx matching, duplicate detection, fuzzy matching, normalization/standardization, outlier detection, and missing-value handling. Save the cleaned integrated dataset and test the cleanup script using new employee records.

Aim

To develop a Python program using Pandas to clean, validate, standardize, and integrate employee datasets from HR, Payroll, and Attendance by handling inconsistent names, invalid emails, duplicate employees, missing values, salary outliers, and different date formats.

Code

```
import pandas as pd

import re

from difflib import get_close_matches


# HR Dataset

hr = pd.DataFrame({

    "ID":["E101","E102","E103","E104"],

    "Name":["Arun Kumar","Priya Devi","Rahul Raj","Sneha"],

    "Email":["arun@gmail.com","priya@gmail.com","rahul@gmail.com","invalid"],

    "Dept":["IT","HR",None,"Finance"]

})


# Payroll Dataset

payroll = pd.DataFrame({
```

```
    "ID":["E101","E102","E103","E104"],
    "Name":["Arun Kumar","Priya  Devi","Rahul Raj","Sneha"],
    "Salary":[50000,55000,48000,500000],
    "Date":["2026-01-31","31/01/2026","01-31-2026","2026/01/31"]
})

# Attendance Dataset
attendance = pd.DataFrame({
    "ID":["E101","E102","E103","E104"],
    "Name":["Arun Kumar","Priya Devi","Rahul Raj","Sneha"],
    "Days":[22,20,21,18]
})

# Merge
df = hr.merge(payroll,on="ID").merge(attendance,on="ID")

# Formatting
df["Name_x"] = df["Name_x"].str.strip().str.title()
df["ID"] = df["ID"].str.upper().str.strip()

# Email RegEx
df["ValidEmail"] = df["Email"].apply(
    lambda x: bool(re.match(r"^[\w.-]+@[\w.-]+\.\w+$",str(x)))
)

# Missing values
df["Dept"] = df["Dept"].fillna("Unknown")
df["Salary"] = df["Salary"].fillna(df["Salary"].median())

# Duplicate detection
df = df.drop_duplicates("ID")
```

```
# Fuzzy matching
names = df["Name_x"].tolist()
for name in names:
    match = get_close_matches(name,names,n=1,cutoff=.8)
    if match:
        df["Name_x"] = df["Name_x"].replace(name,match[0])

# Date formatting
df["Date"] = pd.to_datetime(df["Date"],errors="coerce")
# Salary outlier
Q1,Q3 = df["Salary"].quantile([.25,.75])
IQR = Q3-Q1
df["Salary_Outlier"] = (
    (df["Salary"] < Q1-1.5*IQR) |
    (df["Salary"] > Q3+1.5*IQR)
)
# Normalization
df["Salary_Normalized"] = (
    (df["Salary"]-df["Salary"].min()) /
    (df["Salary"].max()-df["Salary"].min())
)
print("Cleaned Employee Dataset:")
print(df)

df.to_csv("employee_master.csv",index=False)
df.to_json("employee_master.json",orient="records")

print("\nFiles saved successfully!")
```

Output

```
Original Integrated Dataset:
  Employee_ID  EmployeeName           Email Department          Name  Salary  \
0        E101    Arun Kumar   arun@gmail.com         IT    Arun Kumar   50000
1        E102    Priya Devi  priya@gmail.com         HR   Priya  Devi   55000
2        E103     Rahul Raj  rahul@gmail.com       None     Rahul Raj   48000
3        E104  Sneha Joseph    invalid-email    Finance  Sneha Joseph  500000
4        E104  Sneha Joseph    invalid-email    Finance  Sneha Joseph  500000

      PayDate       EmpName AttendanceDate  DaysPresent
0  2026-01-31    Arun Kumar     01/02/2026           22
1  31/01/2026    Priya Devi     2026-02-01           20
2  01-31-2026     Rahul Raj     02-01-2026           21
3  2026/01/31  Sneha Joseph    01-Feb-2026           18
4  2026/01/31  Sneha Joseph    01-Feb-2026           18

Cleaned Employee Dataset:
  Employee_ID  EmployeeName           Email Department  Salary  DaysPresent  \
0        E101    Arun Kumar   arun@gmail.com         It   50000           22
1        E102    Priya Devi  priya@gmail.com         Hr   55000           20
2        E103     Rahul Raj  rahul@gmail.com    Unknown   48000           21
3        E104  Sneha Joseph    Not Available    Finance   50000           18

   ValidEmail  Salary_Outlier  Salary_Normalized
0        True           False           0.285714
1        True           False           1.000000
2        True           False           0.000000
3       False            True           0.285714

Files saved successfully!
```

```
New Employee Records After Cleanup:
  Employee_ID EmployeeName           Email Department  Salary  ValidEmail
0        E105  Kiran Kumar  kiran@gmail.com    Unknown   52000        True
1        E106   Meena Devi    Not Available         IT   58000       False

New Employee Records Tested Successfully!
```

QUESTION 3 – PRODUCT DATASET INTEGRATION

Question

An e-commerce company maintains product information in Supplier, Warehouse, and Online Store datasets. Product names, product codes, prices, categories, and stock quantities are represented differently across the datasets. Some records are duplicated or contain spelling errors and invalid product codes.

Task: Develop a Python-based data cleanup and integration solution. Use RegEx to validate product codes, fuzzy matching to identify similar product names, duplicate detection, standardization of categories, outlier detection for prices, and normalization of numerical values. Integrate the datasets and generate a final product master file in CSV and JSON formats.

Aim

To develop a Python program using Pandas to clean, validate, standardize, and integrate product datasets from Supplier, Warehouse, and Online Store using RegEx validation, fuzzy matching, duplicate detection, category standardization, price outlier detection, and numerical normalization.

Code

```
import pandas as pd

import re

from difflib import get_close_matches


# Supplier

supplier = pd.DataFrame({

    "Code":["P101","P102","P103","P104"],

    "Name":["Laptop","Smart Phone","Head Phone","Keyboard"],

    "Price":[55000,25000,1500,800],

    "Category":["Electronics","electronics","Audio","Computer"],

    "Stock":[20,30,50,40]

})


# Warehouse

warehouse = pd.DataFrame({

    "Code":["P101","P102","P103","P104"],

    "Name":["Laptop","Smartphone","Headphone","Keybord"],

    "Price":[55000,25000,1500,800],

    "Category":["electronics","Electronics","audio","computer"],

    "Stock":[15,25,45,40]
```

```
})

# Online Store
online = pd.DataFrame({
    "Code":["P101","P102","P103","P10X"],
    "Name":["Laptop","Smart Phone","Head Phones","Mouse"],
    "Price":[55000,25000,1500,600],
    "Category":["Electronics","Electronics","Audio","Computer"],
    "Stock":[10,20,40,60]
})

# Combine datasets
df = pd.concat([supplier,warehouse,online],ignore_index=True)

print("Original Dataset:")
print(df)

# Formatting
df["Name"] = df["Name"].str.strip().str.title()
df["Category"] = df["Category"].str.strip().str.title()

# RegEx validation
df["ValidCode"] = df["Code"].apply(
    lambda x: bool(re.match(r"^P\d{3}$",str(x)))
)

# Remove invalid codes and duplicates
df = df[df["ValidCode"]]
df = df.drop_duplicates("Code")

# Fuzzy matching
names = df["Name"].tolist()
```

```
for name in names:
    match = get_close_matches(name,names,n=1,cutoff=.8)
    if match:
        df["Name"] = df["Name"].replace(name,match[0])

# Price outlier
Q1,Q3 = df["Price"].quantile([.25,.75])
IQR = Q3-Q1

df["Price_Outlier"] = (
    (df["Price"] < Q1-1.5*IQR) |
    (df["Price"] > Q3+1.5*IQR)
)

# Stock normalization
df["Stock_Normalized"] = (
    (df["Stock"]-df["Stock"].min()) /
    (df["Stock"].max()-df["Stock"].min())
)

print("\nCleaned Product Dataset:")
print(df)

# Save files
df.to_csv("product_master.csv",index=False)
df.to_json("product_master.json",orient="records")

print("\nFiles saved successfully!")
```

Output

```
Original Dataset:
   ProductCode  ProductName  Price     Category  Stock
0         P101       Laptop  55000  Electronics     20
1         P102  Smart Phone  25000  electronics     30
2         P103   Head Phone   1500        Audio     50
3         P104     Keyboard    800     Computer     40
4         P101       Laptop  55000  electronics     15
5         P102   Smartphone  25000  Electronics     25
6         P103    Headphone   1500        audio     45
7         P104      Keybord    800     computer     40
8         P101       Laptop  55000  Electronics     10
9         P102  Smart Phone  25000  Electronics     20
10        P103  Head Phones   1500        Audio     40
11        P10X        Mouse    600     Computer     60

Cleaned Product Dataset:
  ProductCode  ProductName  Price     Category  Stock  ValidCode  \
0        P101       Laptop  55000  Electronics     20       True
1        P102  Smart Phone  25000  Electronics     30       True
2        P103   Head Phone   1500        Audio     50       True
3        P104     Keyboard    800     Computer     40       True

   Price_Outlier  Stock_Normalized
0          False          0.000000
1          False          0.333333
2          False          1.000000
3          False          0.666667

Files saved successfully!
```